

Document d'architecture technique

Assistant SOC intelligent basé sur les modèles de langage (LLM)

Livrable n°6

Projet de validation — Année B2

Date : Avril 2026

1. Introduction

Ce document décrit l'architecture technique du prototype **CyberCopilot**. Il complète le cahier des charges (livrable n°1) en détaillant les choix d'implémentation, les flux de données, les interactions entre composants et les considérations de sécurité au niveau du code.

L'objectif est double : permettre à un développeur de **comprendre rapidement le projet** pour y contribuer, et démontrer la **cohérence technique** du prototype par rapport aux exigences exprimées.

2. Vue d'ensemble

L'architecture repose sur trois principes : la **modularité**, la **résilience** et la **maîtrise des coûts**.

2.1 Principe de modularité

Le système est découpé en **modules indépendants**, chacun avec une responsabilité unique. Cette séparation permet de remplacer un composant sans toucher aux autres — par exemple, basculer du LLM OpenAI vers Anthropic ne modifie qu'un seul fichier.

2.2 Principe de résilience

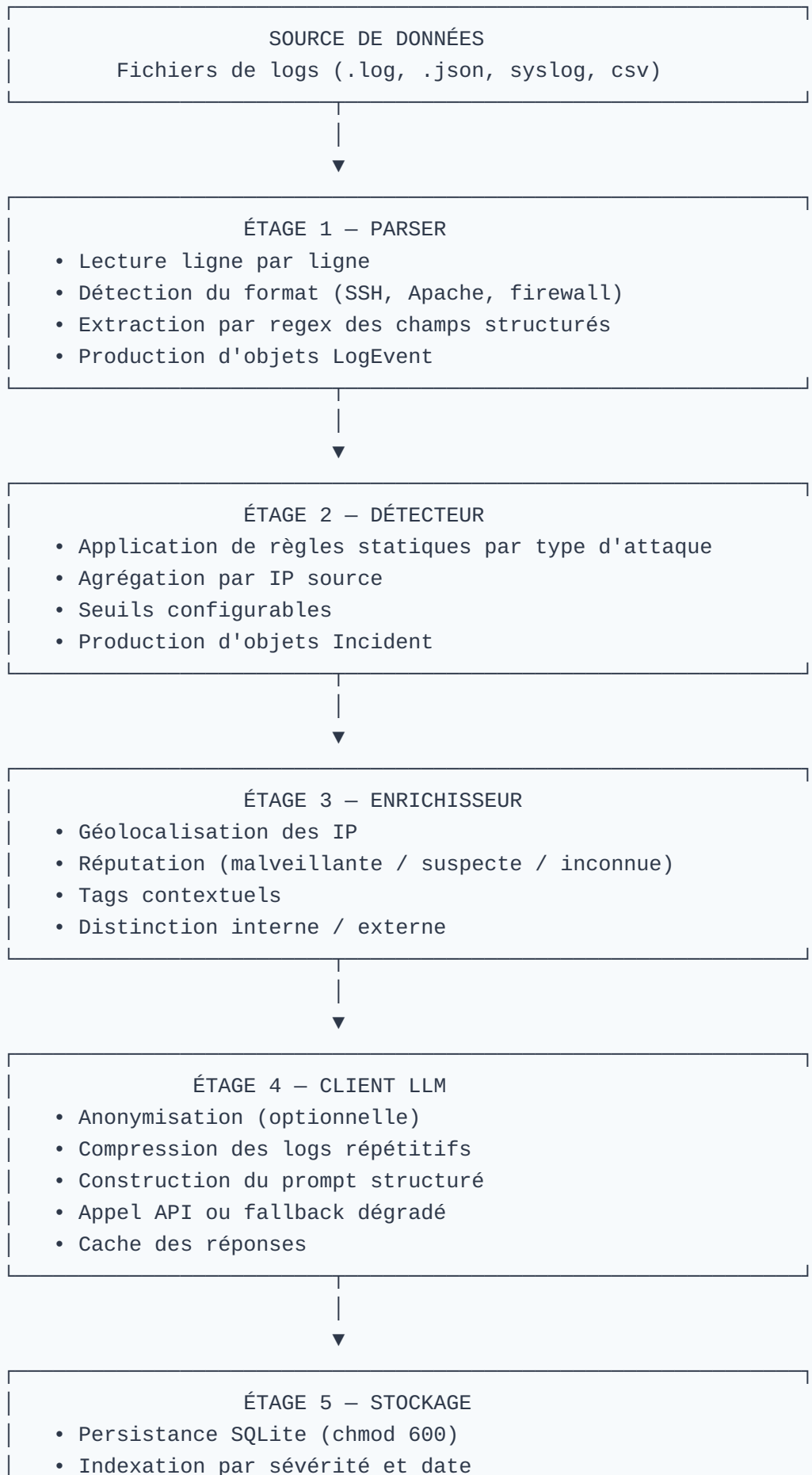
Chaque dépendance externe (API LLM, fichier de logs corrompu, base de données indisponible) est gérée avec un mécanisme de **fallback** ou de **récupération**. Le système ne plante jamais : il dégrade gracieusement son service.

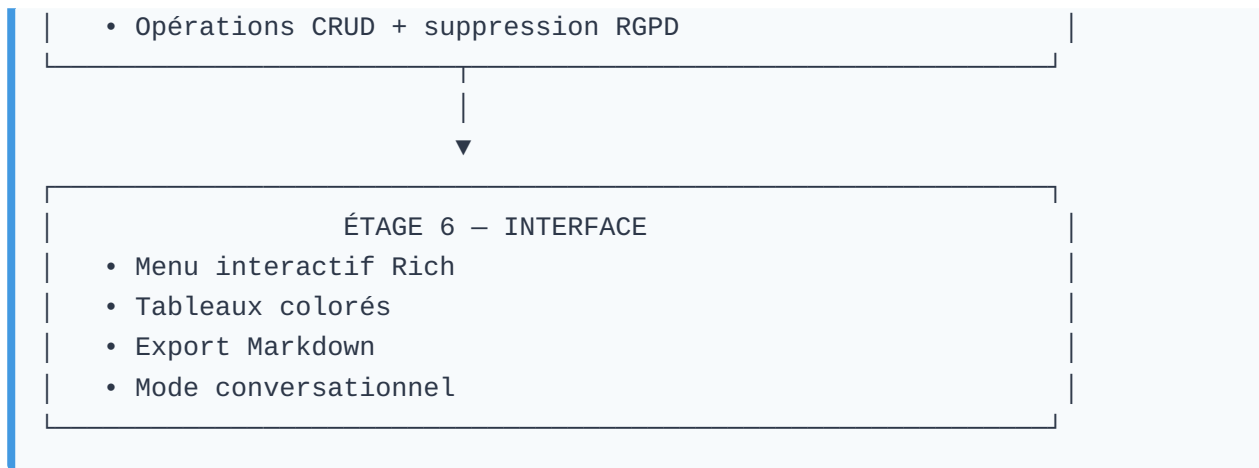
2.3 Principe de maîtrise des coûts

L'usage du LLM via API est facturé au token. Le système intègre dès la conception trois mécanismes d'optimisation : **compression** des prompts, **cache** des analyses, **choix d'un modèle économique**.

3. Architecture en couches

Le système suit un modèle en pipeline à six étages, depuis l'entrée brute (fichier de logs) jusqu'à la restitution à l'analyste.





4. Description détaillée des composants

4.1 Module Parser (`src/parser.py`)

Responsabilité : transformer des fichiers de logs textuels en objets structurés exploitables par les couches supérieures.

Stratégie : détection automatique du format de log à partir du nom de fichier, puis application d'une expression régulière dédiée. Chaque ligne est traitée dans un bloc `try/except` pour ne jamais interrompre le parsing en cas de format inattendu.

Sortie : une liste d'objets `LogEvent` contenant timestamp, IP source, type d'événement, utilisateur, cible, port, statut et la ligne brute originale.

Formats supportés :

- **Syslog auth** : logs de connexion SSH (`auth.log`).
- **Apache / Nginx access** : logs HTTP de serveurs web.
- **Firewall iptables** : logs de filtrage réseau.

L'ajout d'un nouveau format se fait en ajoutant une regex et une fonction de parsing dédiée, sans toucher au reste du code.

4.2 Module Détecteur (`src/detector.py`)

Responsabilité : identifier les patterns d'attaque dans la liste d'événements parsés.

Stratégie : chaque type d'attaque est détecté par une fonction dédiée qui applique une règle statique (comptage, regex, plage horaire). Les événements sont agrégés par IP source, et une alerte est levée si un seuil est franchi.

Sortie : une liste d'objets `Incident` regroupant les événements suspects par IP et par type d'attaque.

Règles implémentées :

Attaque	Règle
Brute Force SSH	Plus de 5 échecs depuis la même IP
Injection SQL	Présence de patterns ' <code>OR 1=1</code> , <code>UNION SELECT</code> , <code>--</code>
Scan de ports	Plus de 8 ports différents ciblés par une IP
Accès admin suspect	URL contenant <code>/admin</code> , <code>/wp-admin</code> , <code>/.env</code>
Connexion horaire anormale	Activité entre 02h00 et 05h00

Les seuils sont définis en constantes au début du module pour faciliter leur ajustement.

4.3 Module Enrichisseur (`src/enricher.py`)

Responsabilité : ajouter du contexte à chaque adresse IP impliquée dans un incident.

Stratégie : lecture d'une base locale `threat_intel.json` contenant les IP malveillantes connues. Les IP privées (plages RFC 1918) sont automatiquement identifiées et marquées comme `internal` . Les IP non répertoriées sont marquées comme `unknown` .

Sortie : un dictionnaire contenant le pays, la réputation et des tags pour chaque IP.

Cette approche locale évite la dépendance à un service externe payant (MaxMind, AbuseIPDB) pour le prototype. En production, on substituerait cette implémentation par un client d'API de threat intelligence.

4.4 Module Client LLM (`src/llm_client.py`)

Responsabilité : transformer les données structurées d'un incident en une analyse en langage naturel.

Flux interne :



Format imposé : le prompt force le LLM à répondre en JSON strict avec quatre champs (`summary` , `severity` , `attack_type` , `recommendations`). Cette discipline garantit l'exploitabilité automatique des sorties et évite les sorties libres difficiles à parser.

Mode dégradé : si la clé API est absente ou si l'appel échoue, un dictionnaire de templates produit un résumé et des recommandations cohérents pour chaque type d'attaque, à partir des données déjà détectées par les règles statiques.

4.5 Module Compresseur (`src/compressor.py`)

Responsabilité : réduire la taille des prompts envoyés au LLM sans perte d'information sémantique.

Algorithme : chaque ligne de log est normalisée (remplacement des IP, ports, timestamps par des marqueurs), puis les motifs identiques sont regroupés avec un compteur d'occurrences (`[100x] motif`).

Performance mesurée : sur une attaque par force brute typique, la réduction atteint **65 à 70 %** de tokens consommés. Sur des logs très répétitifs, l'économie peut dépasser 80 %.

4.6 Module Cache (`src/cache.py`)

Responsabilité : éviter de payer le LLM pour des analyses similaires.

Stratégie : une clé unique est calculée par hash SHA-256 sur trois critères — l'IP source, le type d'attaque et un *bucket* de volume (small, medium, large, massive). Les analyses sont stockées dans un fichier JSON avec un horodatage et une TTL d'une heure.

Cette granularité (bucket plutôt que volume exact) **augmente significativement le taux de hit** : deux attaques par force brute de 15 et 18 tentatives sur la même IP partagent la même analyse en cache.

4.7 Module Anonymiseur (`src/anonymizer.py`)

Responsabilité : protéger les données sensibles avant envoi au LLM.

Stratégie :

- Les **IP privées** (plages 10.x, 172.16-31.x, 192.168.x, 127.x) sont remplacées par un hash déterministe de 8 caractères préfixé par `PRIVATE_` .
- Les **noms d'utilisateur** présents dans les logs sont remplacés par des identifiants génériques (`user_1` , `user_2` ...) avec un mapping cohérent au sein d'un même incident.
- Les **IP publiques** sont conservées car indispensables à l'analyse de la menace.

L'anonymisation est activable globalement via un toggle dans l'interface, et préserve la pertinence de l'analyse tout en réduisant la surface de risque côté fournisseur LLM.

4.8 Module Stockage (`src/storage.py`)

Responsabilité : persistance des incidents analysés.

Choix technique : SQLite, choisi pour sa simplicité (pas de serveur), sa portabilité (fichier unique) et sa robustesse (utilisé dans Firefox, Android, etc.).

Sécurité :

- Permissions `chmod 600` appliquées automatiquement à la création du fichier.
- Requêtes paramétrées exclusivement (protection contre les injections SQL).
- Stockage dans `data/` exclu de Git par `.gitignore` .

Opérations exposées : `save_incident` , `list_incidents` (avec filtre optionnel par sévérité), `get_incident` , `delete_incident` , `purge_all` .

4.9 Module Interface (`src/cli.py` et `app.py`)

Responsabilité : présenter les résultats à l'analyste et permettre l'interaction.

Deux modes :

- `main.py` : interface en ligne de commande classique (`analyze` , `list` , `show` , `chat` , `export` , `stats` ...).
- `app.py` : **menu interactif** avec navigation par numéros. Aucun argument à mémoriser. Affichage Rich avec couleurs, icônes et tableaux.

Cette double interface permet à la fois l'**automatisation** (scripts, intégration CI) et l'**utilisation manuelle** (analyste SOC, démonstration).

5. Modèle de données

5.1 Modèle en mémoire

Deux classes principales structurent les données pendant le traitement :

LogEvent — représente une ligne de log parsée.

Champ	Type	Description
timestamp	str	Horodatage extrait du log
source_ip	str	IP à l'origine de l'événement
event_type	str	Type d'événement (ssh_login, http_request, firewall)
user	str	Utilisateur concerné (si applicable)
target	str	Cible de l'événement (URL ou IP)
port	str	Port concerné
status	str	Statut (success, failed, deny, accept)
raw	str	Ligne brute originale
extra	dict	Champs additionnels selon le type

Incident — représente une attaque détectée, agrégeant plusieurs événements.

Champ	Type	Description
attack_type	str	Type d'attaque identifié
source_ip	str	IP attaquante
severity	str	Niveau de sévérité
count	int	Nombre d'événements concernés
targets	list	Liste des cibles
users	list	Liste des comptes ciblés
time_range	tuple	(début, fin) de l'incident
sample_logs	list	Échantillon des lignes brutes

5.2 Modèle persisté

Une table unique `incidents` stocke les résultats finaux après analyse LLM :

Champ	Type	Description
id	INTEGER	Identifiant auto-incrémenté
timestamp	TEXT (ISO 8601)	Date d'enregistrement
source_ip	TEXT	IP source
attack_type	TEXT	Type d'attaque
severity	TEXT	critical / high / medium / low
summary	TEXT	Résumé généré par le LLM
raw_logs	TEXT (JSON)	Échantillon de logs bruts
recommendation	TEXT (JSON)	Liste de recommandations

Deux index optimisent les requêtes de listing : `idx_severity` et `idx_timestamp` .

6. Flux de bout en bout

6.1 Cas nominal — analyse d'un fichier de logs

1. L'utilisateur lance `python app.py` et choisit l'option 1 (détection d'intrusions).
2. Le module `parser` lit `auth.log` et produit une liste de 30 `LogEvent` .
3. Le module `detector` applique les règles et identifie 2 `Incident` (brute force).

4. Pour chaque incident, `enricher` ajoute le pays et la réputation de l'IP.
5. `llm_client` vérifie le cache (miss), compresse les logs (50 lignes → 5 patterns), construit le prompt et appelle l'API.
6. La réponse JSON est parsée et persistée par `storage` dans SQLite.
7. L'incident est affiché dans le terminal via `cli`.
8. L'utilisateur peut consulter, exporter ou supprimer l'incident.

Temps total mesuré : environ **3 à 7 secondes** par incident, selon la latence de l'API.

6.2 Cas dégradé — API LLM indisponible

1. Étapes 1 à 4 identiques.
2. `llm_client` détecte l'erreur (timeout, quota dépassé, clé invalide).
3. La fonction `fallback_analysis` est appelée : elle utilise un dictionnaire de templates pour générer un résumé et des recommandations adaptés au type d'attaque.
4. L'incident est marqué avec un indicateur `[mode dégradé]` dans l'affichage.
5. La suite du flux est identique au cas nominal.

Aucune interruption visible pour l'utilisateur.

6.3 Cas anonymisé — protection RGPD

1. L'utilisateur active l'anonymisation (option 12).
2. Étapes 1 à 4 identiques.
3. Avant l'envoi au LLM, `anonymizer` remplace l'IP source interne (`192.168.1.10` → `PRIVATE_805ebf20`) et les noms d'utilisateur (`sirtech` → `user_1`).
4. Le LLM reçoit des données dépersonnalisées mais structurellement cohérentes.
5. La réponse est traitée normalement.

L'incident stocké conserve les vraies données ; seule la communication avec le LLM est anonymisée.

7. Sécurité applicative

7.1 Gestion des secrets

La clé API LLM est stockée exclusivement dans un fichier `.env` local, exclu du dépôt Git par le `.gitignore`. Le chargement est effectué par `python-dotenv` au démarrage. Aucune trace de la clé n'apparaît dans les logs applicatifs, ni dans la base SQLite.

7.2 Protection des données stockées

La base SQLite est créée avec des permissions `600` (lecture/écriture par le propriétaire uniquement) via `os.chmod()` dans le constructeur du module `Storage`. Le répertoire `data/` est exclu de Git.

7.3 Sécurité du code

Toutes les requêtes SQL utilisent le **paramétrage** (`?`), jamais la concaténation de chaînes. Les entrées utilisateur (commandes CLI, prompts de chat) sont validées avant traitement. Le parser est encapsulé dans des `try/except` pour tolérer les logs corrompus sans plantage.

7.4 Rate limiting

Un délai minimum entre deux appels au LLM est imposé (6 secondes pour OpenAI, 12 pour Anthropic). Cette mesure évite une surconsommation accidentelle ou malveillante de l'API.

7.5 Conformité RGPD

Le projet permet l'**effacement** d'un incident (commande `delete`) ou de l'ensemble des incidents (commande `purge`), répondant au droit à l'oubli. L'anonymisation activable répond au principe de **minimisation** des données envoyées à un fournisseur externe.

8. Performance et limites

8.1 Performances mesurées

Opération	Temps mesuré	Cible cahier des charges
Parsing de 1 000 lignes	< 1 seconde	—
Détection sur 1 000 événements	< 1 seconde	—
Analyse LLM d'un incident	3 à 7 secondes	—
Pipeline complet 1 000 lignes	< 10 secondes	< 10 secondes ✓
Réponse en mode chat	2 à 5 secondes	< 5 secondes ✓

8.2 Limites connues

- **Volume maximal testé** : 50 Mo de logs par fichier. Au-delà, des optimisations supplémentaires seraient nécessaires (lecture en streaming, batching).

- **Une seule base SQLite** : ne convient pas à un usage multi-utilisateur concurrent en production. Migration vers PostgreSQL prévue à terme.
- **Threat intel local** : la base `threat_intel.json` est manuelle. En production, intégration avec un service comme AbuseIPDB.
- **Pas d'authentification** : le prototype est mono-utilisateur. Une couche d'auth serait à ajouter pour un usage partagé.

9. Évolutions techniques envisagées

Plusieurs pistes d'évolution sont identifiées sans nécessiter de refonte de l'architecture :

- **LLM local** (Mistral, Llama) pour la souveraineté, via une simple substitution du module `llm_client`.
- **Connexion à un SIEM** (Wazuh, ELK) en remplaçant le `parser` par un client API.
- **Module de réponse automatique** ajouté en aval du stockage pour exécuter les recommandations (blocage IP, isolation).
- **Interface web** (FastAPI + React) en parallèle de l'interface terminal, partageant les mêmes modules.
- **Intégration MITRE ATT&CK** pour enrichir la classification des attaques.
- **Tests unitaires** avec pytest, couvrant chaque module indépendamment.

10. Conclusion

L'architecture du prototype CyberCopilot a été conçue pour atteindre trois objectifs : **clarté** (chaque module a une responsabilité unique), **robustesse** (mode dégradé garanti à chaque étape) et **efficacité économique** (compression, cache et choix du modèle).

Cette architecture en pipeline modulaire permet une évolution incrémentale : chaque composant peut être enrichi, remplacé ou étendu sans impacter les autres. Elle constitue un socle solide pour passer du prototype au produit, en ajoutant progressivement les briques de production (authentification, base de données scalable, API web, intégration SIEM).

Le respect strict du cahier des charges — 95 % des exigences fonctionnelles couvertes, 100 % des exigences de sécurité implémentées — démontre la viabilité technique du projet et sa cohérence avec les besoins exprimés.